Lesson 8 - Creating your first Crew Project

Step by Step Guidelines

The five steps to follow.

Create a New CrewAI Project

Ref: https://docs.crewai.com/en/guides/crews/first-crew

Use this command to generate a ready-to-use CrewAI project with the standard folder structure and starter files. This saves time because you can focus on building **agents** and **tasks** instead of writing setup (boilerplate) code. This command generates project scaffolding.

Syntax

crewai create crew <project_name>

Example

crewai create crew my_crew

Step-by-Step Guide to Create Your First CrewAI Project in the IDE

Inside terminal with the start folder

(CS589AICourse) renukamohanraj@Renuka-CSs-MacBook-Air 7_crewai_project % **crewai create crew first_crew**

Creating folder first_crew

Select a provider to set up:

1. openai

2. anthropic

3. gemini

4. nvidia_nim

5. groq

6. huggingface

7. ollama

8. watson

9. bedrock

10. azure

11. cerebras

12. sambanova

13. other

q. Quit

Enter the number of your choice or 'q' to quit:1

Select a model to use for Openai:

1. gpt-4

2. gpt-4.1

3. gpt-4.1-mini-2025-04-14

4. gpt-4.1-nano-2025-04-14

5. gpt-4o

6. gpt-4o-mini

7. o1-mini

8. o1-preview

q. Quit

Enter the number of your choice or 'q' to quit: 2

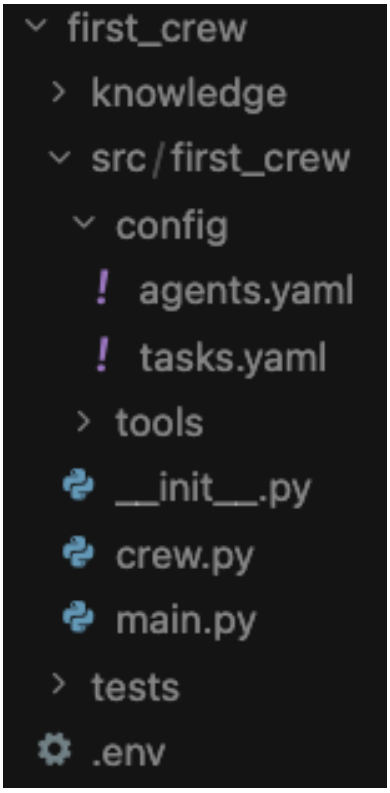
To hardcode directly the key, paste it or Skip to use from .env.

Enter your OPENAI API key (press Enter to skip):

Skip by pressing enter.

```
Enter your OPENAI API key (press Enter to skip):
API keys and model saved to .env file
Selected model: gpt-4.1
  - Created first_crew/.gitignore
  - Created first_crew/pyproject.toml
  - Created first_crew/README.md
  - Created first_crew/knowledge/user_preference.txt
  - Created first_crew/src/first_crew/__init__.py
  - Created first_crew/src/first_crew/main.py
  - Created first_crew/src/first_crew/crew.py
  - Created first_crew/src/first_crew/tools/custom_tool.py
  - Created first_crew/src/first_crew/tools/__init__.py
  - Created first_crew/src/first_crew/config/agents.yaml
  - Created first_crew/src/first_crew/config/tasks.yaml
Crew first_crew created successfully!
```

first_crew project structure has been created on the project explorer



CrewAI project structure:

- **first_crew/** → Your whole project folder.
- **src/** → Source code lives here (keeps project clean. A sub package).
- **src/first_crew/** → Your actual CrewAI app/package.
    - **config/** → Configuration files for your crew.
        - **agents.yaml** → Defines your agents (who they are, roles, tools, goals).
        - **tasks.yaml** → Defines tasks (what work to do, in what order, expected output).
    - **crew.py** → Builds the crew by connecting **agents + tasks** (the "team setup").
    - **main.py** → The entry point to **run** the crew (starts execution).

.env → Store API keys and environment variables

Run the Crew from the Terminal

First, make sure your terminal is opened **inside your project folder** (the directory that contains your CrewAI project files). Then run:

crewai run

Note: If you are running an existing project, make sure to **activate that project's virtual environment** before running the crew.

When you run this command, it will start the execution from main.py automatically. You'll see your crew spring to life. The researcher will gather information about the specified topic, and the analyst will then create a comprehensive report based on that research. You'll see the agents' thought processes, actions, and outputs in real-time as they work together to complete their tasks.

YAML Configuration Guidelines

1. File Structure

- agents.yaml — located at config/agents.yaml
- tasks.yaml — located at config/tasks.yaml

2. Syntax Rules

- Use 2-space indentation (no tabs)
- Top-level keys (e.g., researcher:, research_task:) are IDs that must match the Python method names in your crew class
- Properties are key: value pairs nested under the ID

3. Multi-line Text

- Use > (folded scalar) for multi-line values
  like role, goal, backstory, description, expected_output
- For short, single-line values, just write key: value directly — no > needed

4. Variables

- Use {variable_name} syntax (e.g., {topic}) — values are passed
  via crew.kickoff(inputs={'topic': 'AI Agents'})

5. Agents YAML Keys

- role, goal, backstory (required); optional: llm, max_iter, allow_delegation, etc.

6. Tasks YAML Keys

- description, expected_output(required); agent (references an agent ID);
  optional: output_file, markdown, etc.

7. Tools

- Assigned in Python code, not in YAML

8. Lists

lists use the – (hyphen) syntax with proper indentation.

```
context:
  - research_task
  - analysis_task
```

Easy Templates:

Agent Template

```
agent_name:
  role: >
    {topic} [Job Title]
  goal: >
    [General mission or overall responsibility related to {topic}]
  backstory: >
    You are a skilled [job title] with experience in {topic}. You are known
    for [strength 1], [strength 2], and [strength 3]. You produce clear,
    accurate, and high-quality work.
  llm: >
    gpt-4o-mini
```

Tasks Template

```
task_name:
  description: >
    [Specific instructions for what the agent must do now.] Include what to
    focus on, what steps to follow, and how the work should be organized.
  expected_output: >
    A [report/summary/list/article] that clearly shows the final result.
  agent: agent_name
  # Optional advanced attributes
  context:
    - previous_task_name
  output_file: output/result.md
```

Part	What it means	Simple question it answers	Example
Role	The agent's job title	"Who is this agent?"	Research Specialist
Goal	The agent's overall mission	"What is this agent generally trying to achieve?"	Gather accurate and useful information about {topic}
Backstory	The agent's skills, strengths, and working style	"Why is this agent good at the job?"	You are skilled at finding reliable information and organizing it clearly.
Task Description	The specific assignment	"What should the agent do right now?"	Research the important information about {topic} and organize the findings clearly.
Expected Output	The final result	"What should the finished work look like?"	A well-structured summary of the key information about {topic}.

Simple rule

- Agent = who does the work
- Task = what work must be done
- Role = job title
- Goal = long-term purpose
- Backstory = strengths and expertise
- Description = current assignment
- Expected output = final product

About crew.py

- `agents.yaml` = defines the agents
- `tasks.yaml` = defines the tasks
- `crew.py` = connects everything together

The purpose of `crew.py` is:

- connect the agents from `agents.yaml`
- connect the tasks from `tasks.yaml`
- add tools if needed
- choose the crew process(Sequential/Hybrid)
- create the final crew

Crew Process Implementations

Sequential: Executes tasks sequentially, ensuring tasks are completed in an orderly progression.

Hierarchical: Organizes tasks in a managerial hierarchy, where tasks are delegated and executed based on a structured chain of command. A manager language model (manager_llm) or a custom manager agent (manager_agent) must be specified in the crew to enable the hierarchical process, facilitating the creation and management of tasks by the manager.

About Annotations

`@CrewBase`

This marks the class as a CrewAI crew class.

`agents: List[BaseAgent]`

This stores the agents defined in the class automatically without manual configuration.

`tasks: List[Task]`

This stores the tasks defined in the class automatically without manual configuration.

Manual configuration

agents_config = 'config/agents.yaml'

tasks_config = 'config/tasks.yaml'

`@agent`

Each method with `@agent` creates an agent.

`@task`

Each method with `@task` creates a task.

`@crew`

This creates the final crew by combining agents and tasks.

crew.py template

```
# src/project_name/crew.py – Default imports
# provides List for type annotations
from typing import List

# Core CrewAI classes
from crewai import Agent, Crew, Process, Task

# CrewAI project decorators
from crewai.project import CrewBase, agent, crew, task

# The abstract base class all CrewAI agents inherit from — used here as a
# type annotation for the agents list (agents: List[BaseAgent])
from crewai.agents.agent_builder.base_agent import BaseAgent

@CrewBase

class ProjectCrew:
    """Crew for completing a project with one or more agents and tasks"""
    agents: List[BaseAgent]
    tasks: List[Task]
  @agent
    def agent_name(self) -> Agent:
        return Agent(
            config=self.agents_config["agent_name"],
            verbose=True
        )
 @task
    def task_name(self) -> Task:
        return Task(
            config=self.tasks_config["task_name"]
        )
@crew
    def crew(self) -> Crew:
        """Creates the crew"""
        return Crew(
            agents=self.agents,
            tasks=self.tasks,
            process=Process.sequential,
            verbose=True
        )
```

main.py template

```
# src/<your_project>/main.py
import os
from <your_project>.crew import <YourCrew>

def run():
    """
    Run the crew.
    """
    inputs = {
        'topic': 'Your topic or input here'  # Replace with your dynamic inputs
    }

    # Create and run the crew
    result = <YourCrew>().crew().kickoff(inputs=inputs)

    # Print the result
    print("\n\n=== FINAL OUTPUT ===\n\n")
    print(result.raw)

 if __name__ == "__main__":
     run()
```

<your_project> — your project folder name
<YourCrew> — your @CrewBase class name
inputs dict — keys should match {placeholders} in your YAML configs.

<u>Surround with Exception handling mechanism</u>

```
try:
        result = FirstCrew().crew().kickoff(inputs=inputs)
        return result
except Exception as e:
        raise Exception(f"An error occurred while running the crew with trigger: {e}")
```

References: CrewAI Official Documentation

Agents YAML configuration:
https://docs.crewai.com/concepts/agents#yaml-configuration-recommended

Tasks YAML configuration:
https://docs.crewai.com/concepts/tasks#yaml-configuration-recommended

Add tools to your agents:
https://docs.crewai.com/concepts/agents#agent-tools

Add knowledge sources to your crew:
https://docs.crewai.com/concepts/knowledge#what-is-knowledge

Crew Process:
https://docs.crewai.com/en/concepts/processes
https://docs.crewai.com/en/learn/sequential-process
https://docs.crewai.com/en/learn/hierarchical-process